

How Web Applications Actually Work

Downloadable student notes for Week 4 Lesson 4



DB

By the end of this week, you should be able to:

- Explain the difference between a static site and a dynamic web application.
- Trace a user action from the frontend to the backend and database.
- Identify where validation, authentication, authorization, and sessions belong.
- Describe why roles matter for students, teachers, and admins.
- Explain how domains, reverse proxies, ports, environment variables, and logs fit deployment.

Use these notes before the video, during class, and again before the quiz. Write your own examples in the margins or in a notebook.

Structured Lesson Notes

Read each section slowly. The goal is not memorizing vocabulary; the goal is explaining what happens inside a real application.

1. Static Site vs. Web Application

A static site mostly sends files to the browser. A web application responds to users, talks to a backend, saves data, and changes based on who is signed in.

Concept	Meaning
Static site	Same files for everyone.
Dynamic app	Responses can depend on user, data, and permissions.
Backend	The server-side program that decides what happens.

Browser -> HTML/CSS/JS

Browser -> API -> Database -> API response

Try this: List three LMS screens that must be dynamic instead of static.

2. Frontend: What the User Touches

The frontend is the part in the browser: pages, forms, buttons, layouts, and JavaScript that reacts to clicks. It should be clear, fast, and hard to misuse.

Concept	Meaning
HTML	Structure and content.
CSS	Layout and appearance.
JavaScript	Interaction and data requests.

button click -> collect form values -> `fetch("/api/checkout")`

Try this: Choose one LMS form and describe what the frontend must collect before sending it.

3. Backend: The App Brain

The backend receives requests, checks data, applies rules, talks to the database, and sends a response. Trusted decisions must happen here, not only in the browser.

Concept	Meaning
Route	The URL path handled by the server.
Validation	Checking data before trusting it.
Response	What the server sends back.

```
POST /api/auth/login
GET /api/progress
POST /api/progress/quiz
```

Try this: Explain what the backend should check before saving a quiz result.

4. Database: The App Memory

The backend should save important state in the database: users, enrollments, lesson progress, quiz results, orders, and resources. The browser alone is not enough for teacher/admin visibility.

Concept	Meaning
User table	Who can sign in.
Enrollment table	Who has access.
Progress table	What the student completed.

```
app_users -> course_enrollments -> lesson_progress
```

Try this: Name the tables needed to let an admin approve a student purchase.

5. Login, Sessions, and Roles

Authentication proves who someone is. A session keeps them signed in. Authorization decides what that person is allowed to do.

Concept	Meaning
Authentication	Are you really this user?
Session cookie	Browser proof of a signed-in session.
Role	Student, teacher, or admin permissions.

student: view lessons
teacher: add resources
admin: approve students

Try this: Write one thing a student should never be allowed to do from the frontend.

6. Deployment: Making It Live

Production adds another layer: a domain points to the VPS, a reverse proxy routes the domain to the right app port, environment variables configure secrets, and logs tell you what broke.

Concept	Meaning
Domain	Human-friendly name.
Reverse proxy	Routes web traffic to the right app.
Environment variable	Configuration outside the code.

digitalbridgeacademy.com -> Traefik -> Learning MS :8765

Try this: Trace what happens when a student opens the public LMS domain.

Mini Project

Trace the full LMS flow from student sign-in to quiz submission. Draw or write the steps across browser, frontend JavaScript, backend route, database table, teacher dashboard, and admin oversight.

Submission checklist:

- Use clear labels and short explanations.
- Show the main flow from user action to saved result.
- Include at least one mistake or failure case and how the system should respond.
- Submit the work through the class activity/project area so the teacher can review it.

Quick Review

- What makes a web application different from a static website?
- What should the frontend do before calling an API?
- Why must the backend validate data even if the frontend already checked it?
- What is the difference between authentication and authorization?
- What does a reverse proxy do when more than one app runs on the same VPS?

Study habit: After each topic, write one example from the LMS or a real app you use. That is how the vocabulary becomes practical.